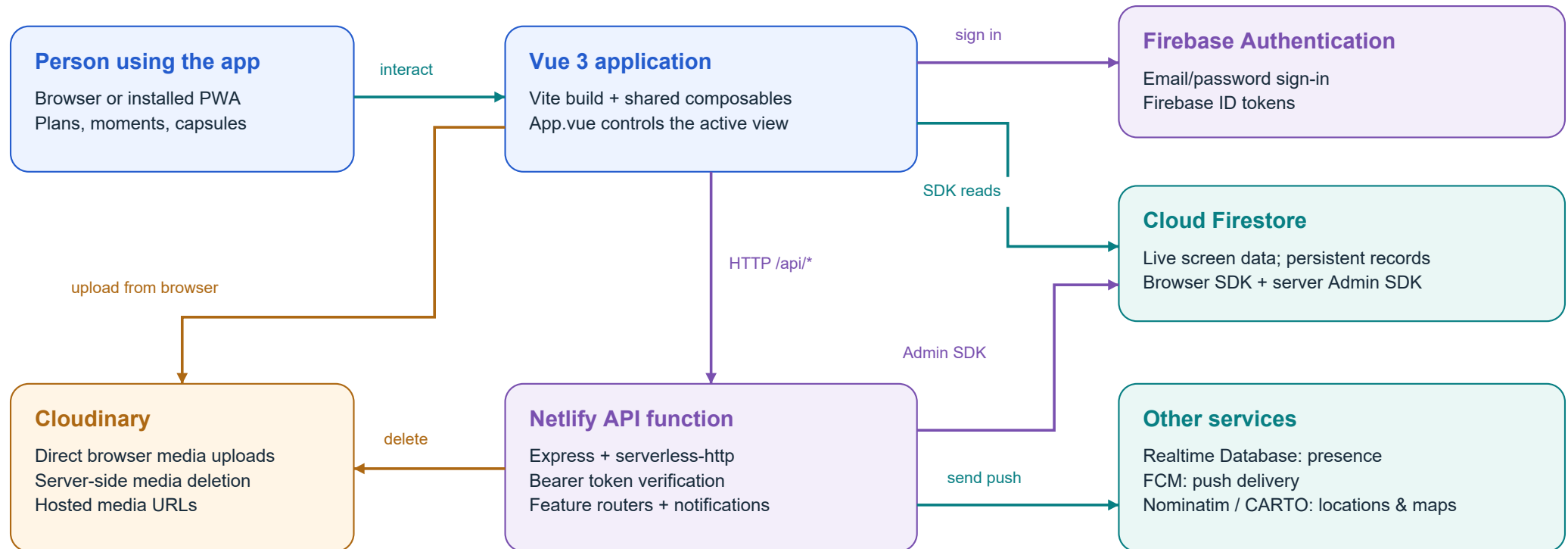


The system at a glance

Vue screens, direct Firebase subscriptions and a Netlify API work together. Arrows show the direction of requests or delivery.

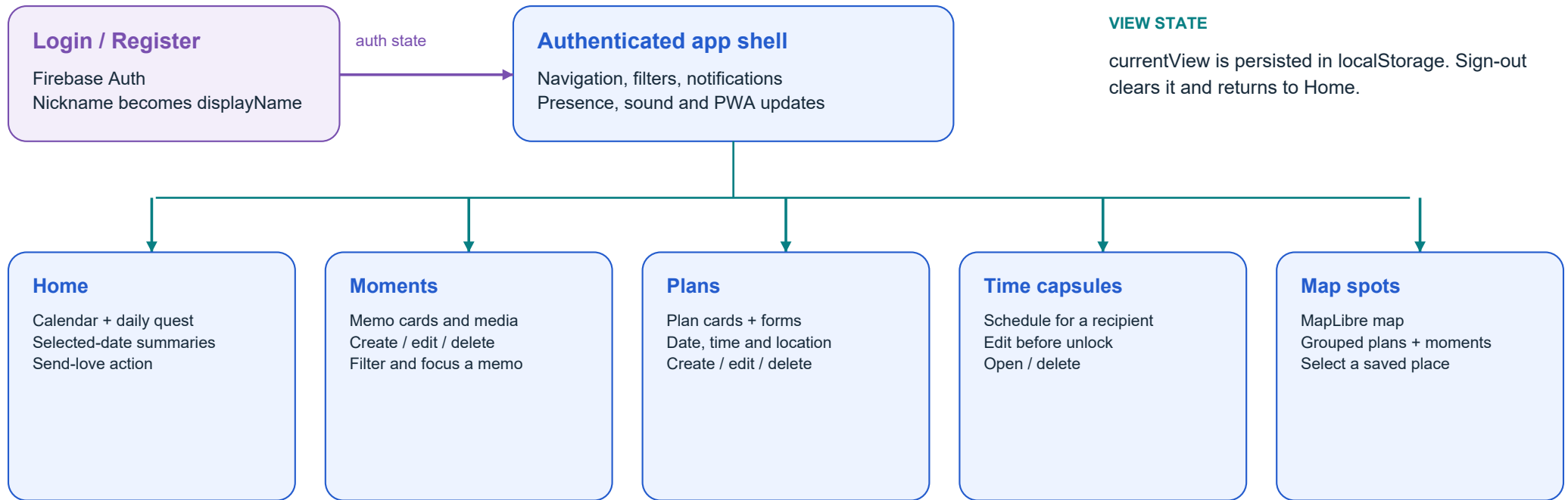


How to read this document

Blue = browser/UI. Purple = identity or API. Teal = stored data and event delivery. Amber = media or external lookup. Solid arrows are calls; dashed arrows are replies or subscriptions. This is a code snapshot, not a verification of deployed cloud settings.

Screens and user interactions

App.vue switches views with local state; there is no Vue Router dependency in the project.



LIVE DATA: FIRESTORE

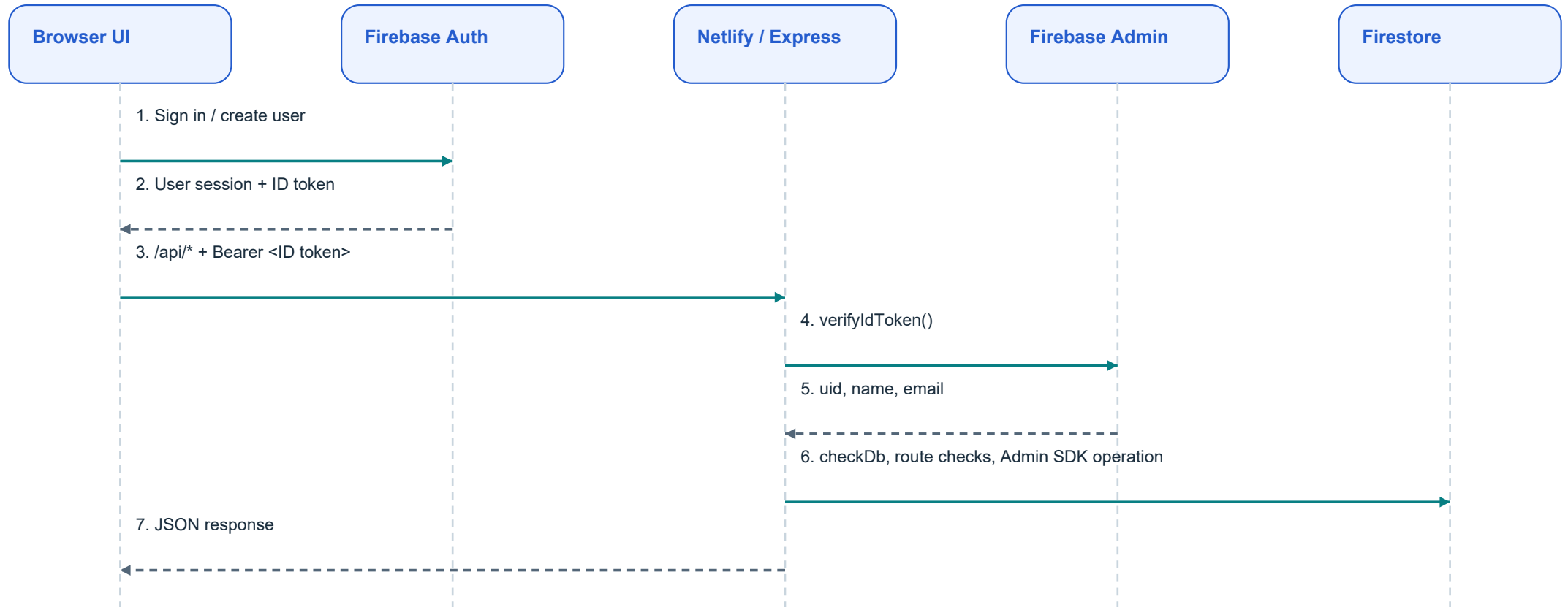
Calendar combines plans, moments, quests and capsules. Map spots derive from plans and moments; there is no separate spots collection.

Notification links reopen the relevant screen

Query parameters such as `?view=plans&planId=...` select the view and focus a record. The parser accepts home, memos, plans and capsules, then removes the consumed parameters from the URL.

Sign-in and API access

Credentials go to Firebase Auth. Application requests carry an ID token in the Authorization header.



Two access-control paths

Browser Firestore calls are governed by `firestore.rules`. Server Admin SDK calls use backend authorization checks. API responses include 401 for a missing token, 403 for an invalid token, and 503 when Firebase or the database is unavailable.

API reference | 16 implemented routes

All routes below require a Firebase Bearer token. JSON bodies use Content-Type: application/json.

VERB	PATH	BODY / INPUT	RESULT / BEHAVIOR
GET	/api/plans	None	Array of plans; date descending
POST	/api/plans	text, date, time, location, coords, hashtags	201: success + planId; sends push
PUT	/api/plans/:planId	Updated plan fields	200: success; sends update push
DELETE	/api/plans/:planId	Path ID	200: success; sends cancellation push
GET	/api/memos	None	Array of memos; createdAt descending
POST	/api/memos	description, date, location, photos, tags	201: success + memoid; sends push
PUT	/api/memos/:memoid	Updated memo fields / photos	200: success; removes discarded media
DELETE	/api/memos/:memoid	Path ID	200: success; removes media + memo
GET	/api/time-capsules	None	success + items; all capsules, unlock order
POST	/api/time-capsules	toUid, unlockAt; title, message, photos	201: success + id; requires future unlock
PUT	/api/time-capsules/:id	title, message, unlockAt, photos	Creator only; unopened and before unlock
POST	/api/time-capsules/:id/open	Path ID	Sender or recipient; at/after unlock; idempotent
DELETE	/api/time-capsules/:id	Path ID	Creator only; deletes capsule + media
POST	/api/quests	date, text	Notification trigger; does not save the quest
POST	/api/register	token (FCM device token)	Stores fcmTokens/{token} with uid
POST	/api/send-love	No body required	Sends a love notification

coords = locationCoords; tags = hashtags. GET routes exist even where screens use Firestore listeners.

Code: [netlify/functions/api.mjs](#); [netlify/functions/routes/{plans,memos,quests,timeCapsules}.mjs](#)

Plans and moments | write, then live refresh

The API persists mutations. Firestore subscriptions update the lists, calendar and map without a manual page reload.



Core payloads

Plan: text, date, time, location, locationCoords, hashtags; creatorUid and createdAt are added by the API.

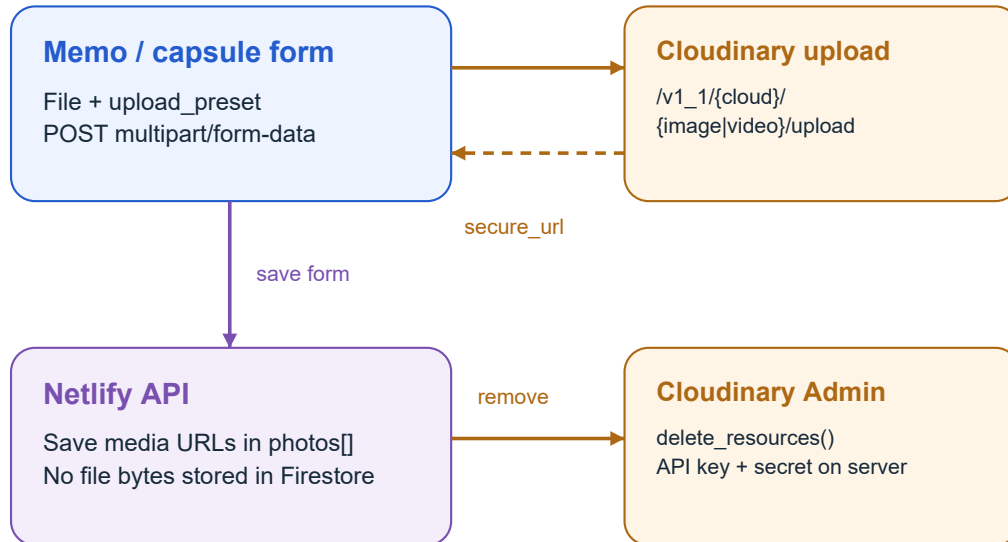
Memo: description, date, location, locationCoords, hashtags, photos[]. Media is uploaded before the record is saved.

Current behavior: plan and memo routes authenticate callers, but do not enforce creator ownership on edits or deletions. Their browser rules also allow any signed-in user to read and write.

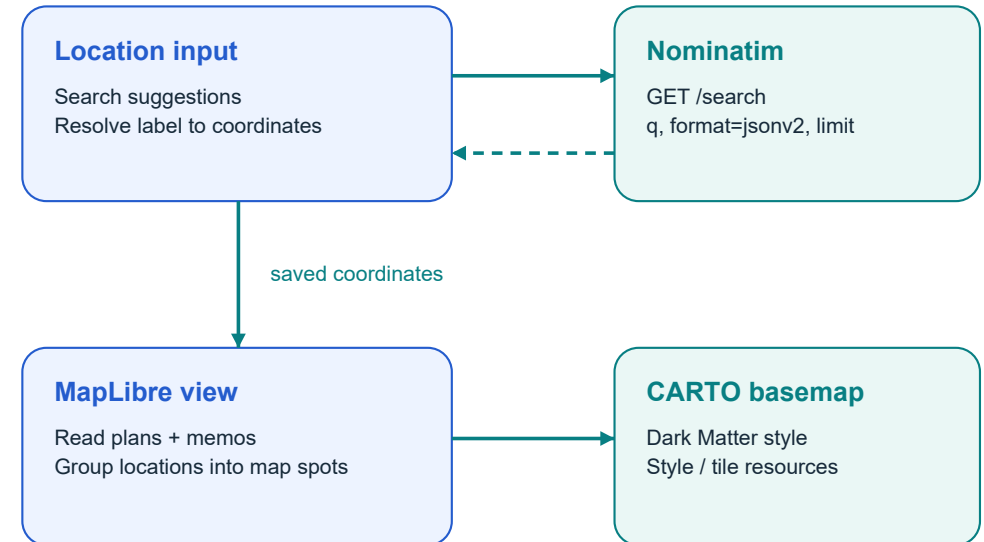
Media and location integrations

Large media files and map lookups travel directly from the browser to external services.

MEDIA



LOCATIONS & MAPS



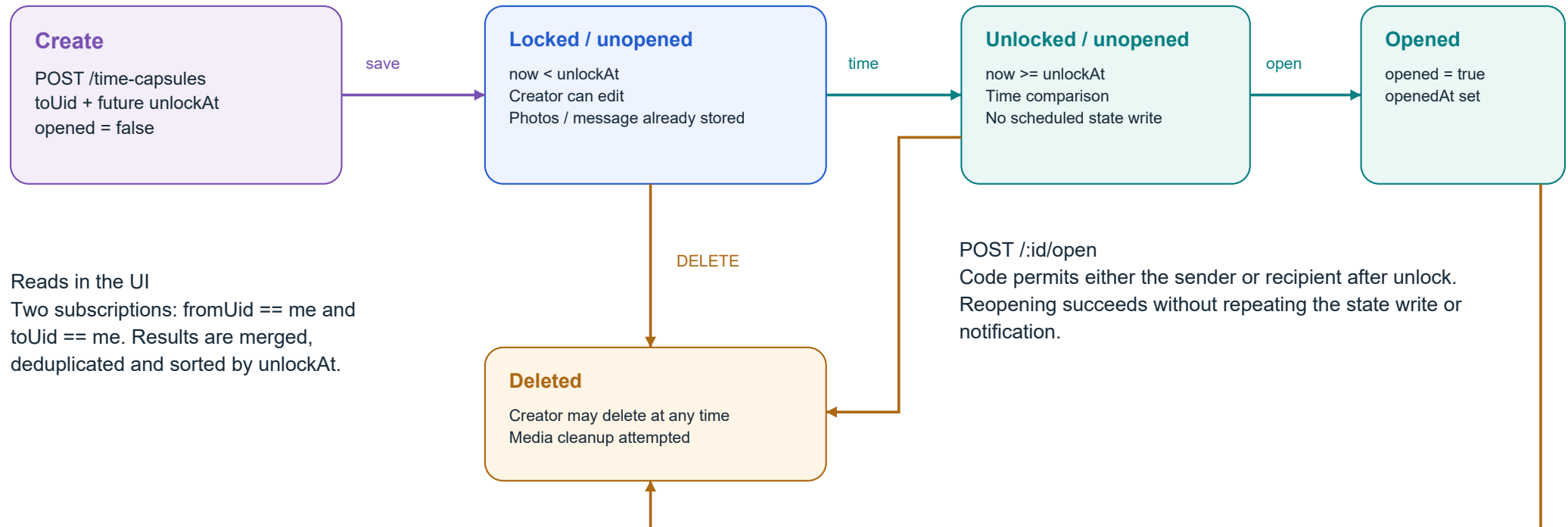
Storage boundary

Firestore holds media URLs and metadata, not the uploaded files. Upload requests use a preset and do not include the Cloudinary API secret. The API uses server credentials when media is removed.

Current behavior: uploads happen before the metadata save. No automatic rollback of a successful upload is shown if saving the record fails. Capsule cleanup catches media-deletion errors; memo cleanup can fail the request.

Time capsules | lifecycle and permissions

A time-based UI state is combined with explicit server checks when a capsule is edited, opened or deleted.

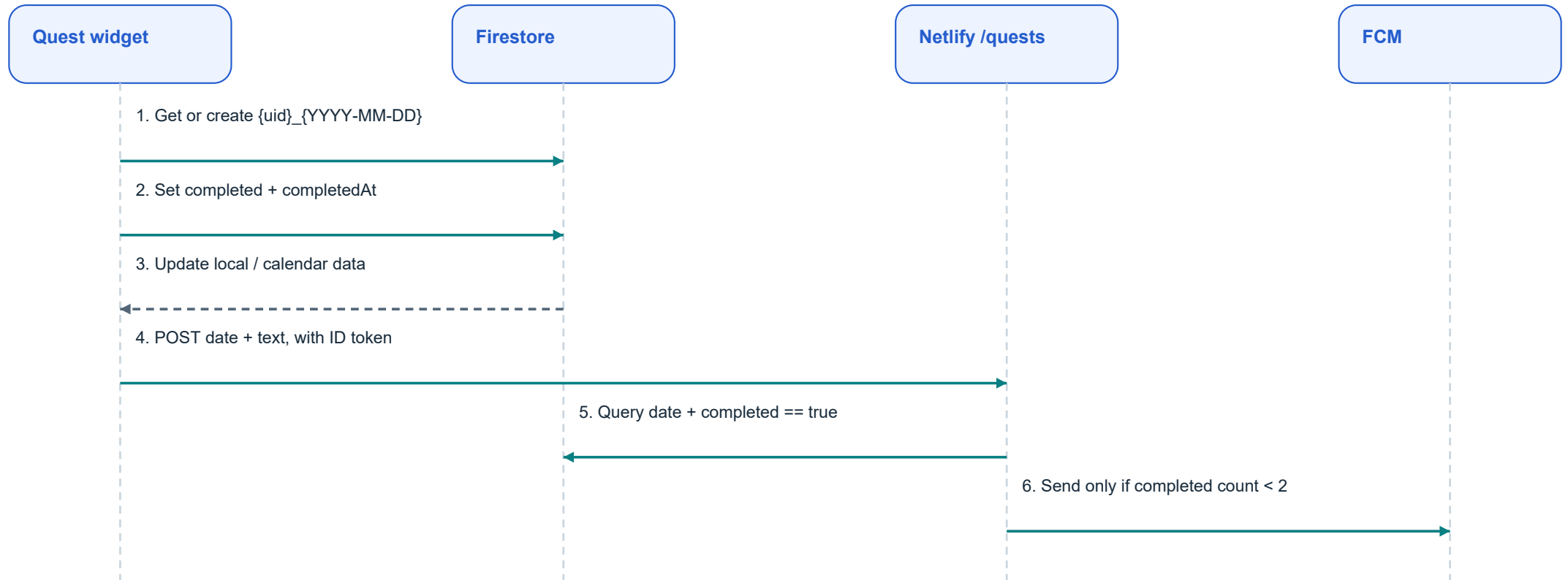


The current lock is not a data-secrecy boundary

Firestore rules let the sender and recipient read complete capsule documents before unlock. The REST GET route returns all capsules to any authenticated caller. The diagram documents this implementation; it does not imply that locked content is withheld by the backend.

Daily quests | client write, server notification

Each user has one quest document per local calendar date. Completion and push delivery are separate operations.

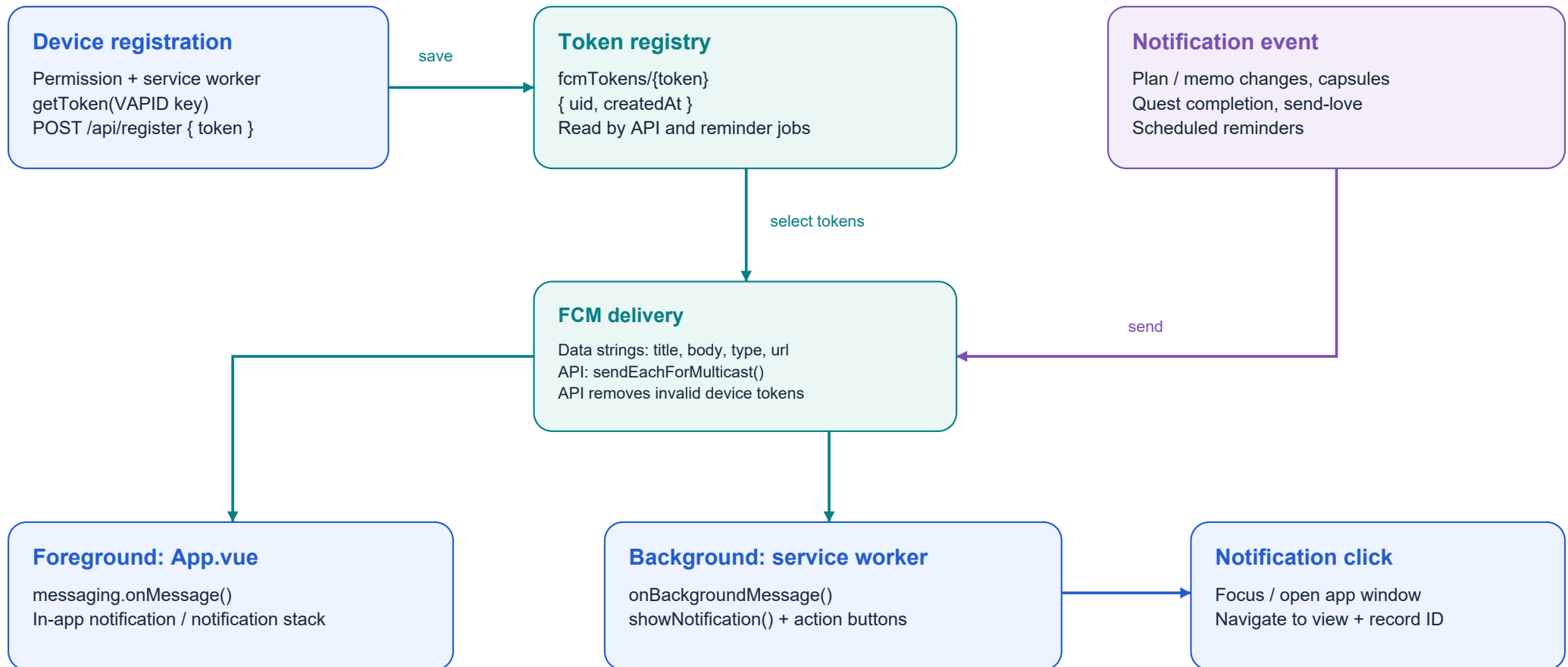


Two-person assumption

The server counts completed quest documents for the date across the collection. At two or more completions it skips the push. It does not create or complete the quest itself. A failed notification request does not roll back the client-side completion.

Push delivery and deep links

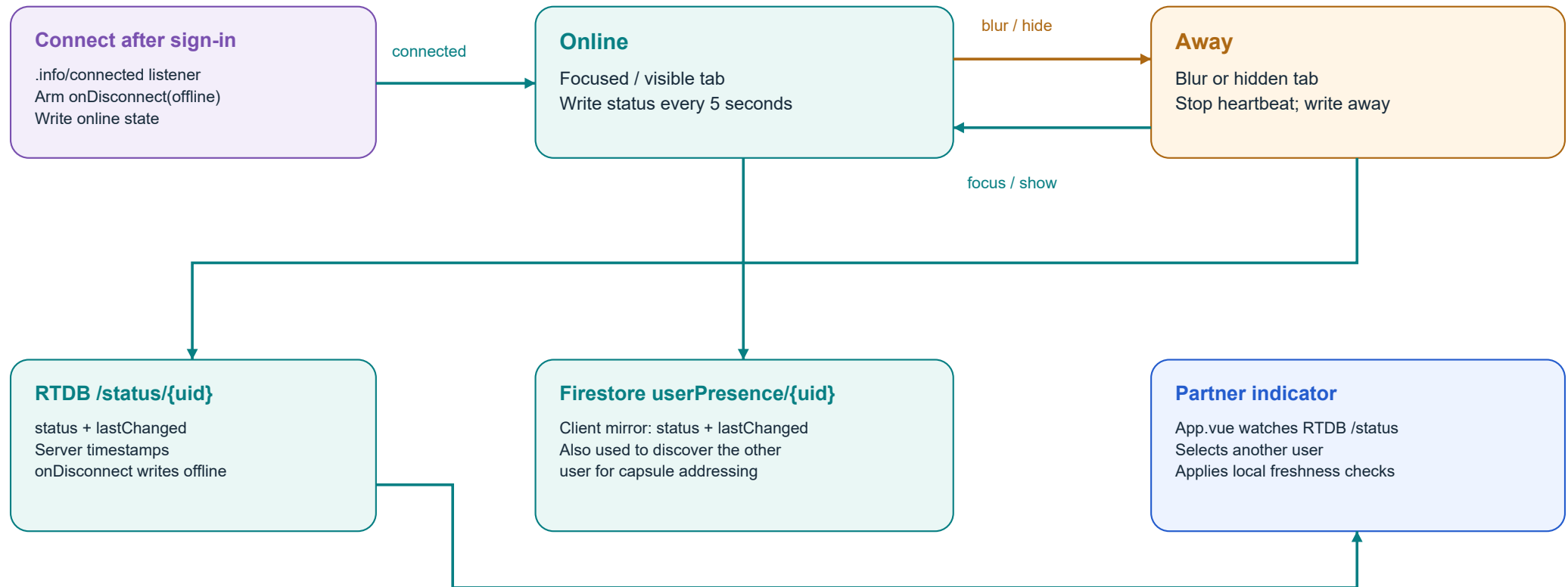
One shared API helper delivers data-only FCM messages; foreground UI and the service worker render them differently.



Recipient behavior: the shared API helper sends to all registered tokens in development; production excludes the initiating UID. It does not filter to a specific capsule toUid. Quest pushes currently carry `/#/calendar`, while the app deep-link parser reads query parameters.

Presence | online, away and offline

Realtime Database is the live presence source. The client also mirrors status into Firestore.



Disconnect handling is asymmetric

The RTDB server can mark a disconnected client offline. The Firestore mirror is written by the client, so it may remain stale after an abrupt disconnect. No server-side presence-mirroring function or Realtime Database rules file is present in this repository.

Data model and access paths

These are document collections and key fields, not relational tables. UID fields refer to Firebase Auth users.

plans/{id}

text, date, time, locationCoords
creatorUid, hashtags, createdAt
Browser: signed-in read/write

memos/{id}

description, date, photos[], locationCoords
creatorUid, hashtags, createdAt
Browser: signed-in read/write

timeCapsules/{id}

fromUid, toUid, unlockAt, message, photos[]
opened, openedAt, unlockDateKey
Browser: participants read; no writes

dailyQuests/{uid_date}

userId, userName, date, text
completed, completedAt, createdAt
Browser: signed-in read; UID write check

userPresence/{uid}

status, lastChanged
Client-written Firestore presence mirror
Browser: signed-in read; own UID write

fcmTokens/{token}

uid, createdAt
Device token is the document ID
Server: register / select / clean up

planReminderNotifications/{key}

planId, reminderCode, dueAt
successCount, failureCount, createdAt
Server: reminder deduplication

anniversaryNotifications/{key}

collectionName, docId, milestoneCode
date, delivery counts, createdAt
Server: anniversary deduplication

RTDB: /status/{uid}

status, lastChanged
Auth UID links to Firestore mirror
Separate database: live presence

Rules and indexes in this repository

No client allow rules are declared for token or reminder-log collections. Capsule indexes combine fromUid + unlockAt and toUid + unlockAt. dailyQuests writes check the resulting userId; deletion uses the same request.resource condition, which has no resulting document.

Scheduled reminders

Two exported Netlify schedules read Firestore, select device tokens and record notification attempts.

PLAN REMINDERS

Every 15 minutes

Cron: */15 * * * *
Read plans + fcmTokens
Default plan time: 09:00

Find due triggers

24 hours before
1 hour before
At the due time

Check dedupe key

planId : reminderCode : dueISO
15-minute trigger window
Skip existing attempt

Send + log

FCM multicast
Write reminder log
Prod excludes plan creator

ANNIVERSARIES

Daily cron

Cron: 0 9 * * *
Read plans + memos
Compare UTC dates

Match milestone

Exactly 6 months later
Exactly 1 year later
Clamp end-of-month dates

Check dedupe key

collection : id : milestone : day
anniversaryNotifications
Skip existing attempt

Send + log

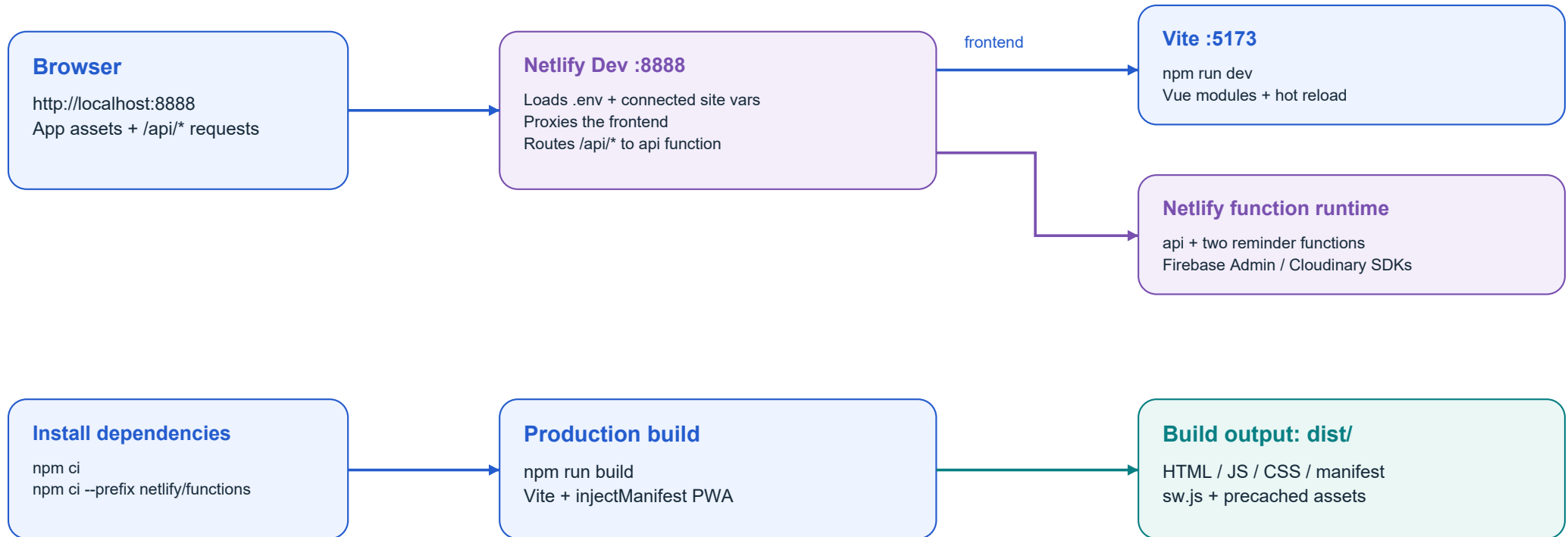
All registered FCM tokens
Record delivery counts
Deep link to plan / memo

Timing and delivery details

Plan due dates use the server runtime timezone; anniversary comparisons use UTC. The log check, push and log write are separate operations, not a transaction. Logged failures are not automatically retried by these handlers. Cloud scheduling was not invoked during documentation.

Local runtime, build and deployment boundaries

Use Netlify Dev for the combined frontend and API. The production frontend is generated into dist/.



Configuration boundaries

VITE_* values configure the browser; Firebase Admin and Cloudinary secrets stay server-side. netlify.toml rewrites /api/* to the API function. firebase.json serves dist/ with an SPA fallback, but does not route /api/* to Netlify.

PWA: Workbox precaches build assets and caches Firebase libraries. The update composable checks every 60 seconds and reloads after a 3-second update notice. No general API/data offline cache is configured.